

Console

A walkthrough of every panel, and what to say about it

The operations console makes five distinct arguments, one per region of the screen. This explains each of them, what the numbers mean, where they look worse than they are, and the questions each panel invites.

NOT PART OF THE SUBMISSION

This document contains rehearsal notes — what to point at, what to say, and how to answer challenges. It is for preparation only and should not go in the submission archive. The technical report (Aegis-Technical-Report.pdf) is the document intended for readers.



Three columns, left to right, in the order the work happens: what arrived, why it was decided, the policy that decided it. The metrics strip runs across the top; the audit trail shares the lower-right panel with model health.

1 Risk appetite — worked through

Watch it work on a real case

APP-31120547AC scored $p = 9.09\%$. The ground truth for that application is **fraud**.

ACTION	EXPECTED COST AT $P = 9.09\%$	COST
Approve	$0.09091 \times 45,000$	₹4,091
Block	$0.90909 \times 1,500$	₹1,364
Review	$200 + 0.09091 \times 0.10 \times 45,000$	₹609

The system said REVIEW — not because 9% "feels borderline", but because reviewing is the cheapest of three priced options. It was fraud, so the review band earned its keep on this case.

The one number that is not pure arithmetic

The panel derives $\tau_{\text{review}} = 0.00494$, but the live threshold is **0.04487** — nine times higher. That is the **analyst-capacity cap**.

The Bayes-optimal threshold assumes unlimited review capacity. At 0.49% it would route an enormous share of traffic to humans. A second constraint caps the review band at the volume the team can actually work (5%), which raises the threshold. It is applied *after* the economics, deliberately, so the trade stays visible instead of being buried in a tuned constant.

It is also why the panel's own footnote says live thresholds are additionally capped: the cap needs a live score distribution, so it happens at serving time rather than in the simulator.

THE DEMO MOMENT — DRAG THE FIRST SLIDER UPWARD

"I have not retrained anything. These thresholds are derived from four business costs by minimising expected loss. Raising the cost of a wrongful block makes the system measurably less willing to block. Changing the bank's risk appetite is arithmetic, not a model release."

The recompute is server-side, calling the same `derive_policy()` the scoring path uses — so what is on screen is the real derivation, not a JavaScript re-implementation that could drift from it.

"Where did ₹45,000 come from?"

It is configuration, not a measurement, and that is the point — it lives in `.env` so a risk owner can set it. A real deployment would use the observed average write-off.

"Why cap review at 5%?"

Because a policy that is optimal on paper and unstaffable in practice is not optimal. It is the difference between a threshold derived in a notebook and one deployable on Monday.

2 Case detail

Ordered the way an analyst actually reads a case: what was decided, why, what it resembles, and what to do.



Header

Application ID, decision pill, and the *calibrated* fraud probability. "Calibrated" matters — an uncalibrated score makes every cost comparison in Section 1 invalid.

Threshold meter

Where this application landed relative to the two live thresholds. The scale is square-root, not linear: review sits at 0.045 and block at 0.217, so on a linear axis every marker would pile against the left edge and show nothing.

Rule override

Appears only when the similarity layer escalated the decision. Shows the model's own verdict alongside the final one.

Ground truth

Replay only. The dataset's true label, so misses are visible as well as catches. The scoring path never sees this.

Analyst briefing

Plain-English summary of the attributions, tagged GEMINI or TEMPLATE.

Principal reasons

Adverse-action codes. Shown only for REVIEW and BLOCK.

Factor contributions

The SHAP force bars — Section 3.

Resembles confirmed fraud

Nearest cases in the vector index, with a strength label.

Record verdict

Confirm fraud / clear as genuine. This is the feedback loop.

The case above is a REVIEW at 5.7% whose ground truth is *genuine* — a needless review. Worth showing rather than hiding: the review band's cost is real, and it is paid on cases like this one.

2 Case detail — the analyst briefing

A plain-English paragraph summarising the attributions, written **after** the decision, from numbers that already exist:

Decision: REVIEW. The strongest driver was match between applicant name and email address (0.12). Also contributing: internal credit risk score (267); session kept alive during application (0). Offsetting this, device operating system (linux) reduced the assessed risk.

The tag in the corner reads GEMINI or TEMPLATE. During rehearsal it will often say TEMPLATE, because the free-tier quota is exhausted and the deterministic fallback is doing the work.

THE MOST IMPORTANT BOUNDARY IN THE SYSTEM

Gemini receives the computed SHAP values and turns them into prose. It **cannot alter a score, a threshold, or an outcome**. Two consequences worth saying out loud:

It is off the *decision* path — narration costs ~3.4s against an 86ms decision, so the stream never waits for it; it is fetched only when a case is opened. And it is off the *correctness* path — a decision that can only be explained when a third-party API is reachable is not auditable, so the template always exists.

The briefing and the reason codes are different things

Worth not conflating, because a sharp question will separate them.

	AUDIENCE	SOURCE
Analyst briefing	The analyst working the case	LLM prose, or a deterministic template
Principal reasons	The declined customer	The SHAP ordering, verbatim

Under Regulation B a declined applicant is entitled to the *specific principal reasons*. Those come straight from the top-four risk factors by attribution magnitude — never paraphrased, never LLM-generated. That is what makes them genuinely principal rather than a generic list.

IF ASKED WHY THE LLM IS NOT MAKING DECISIONS

"Because then the decision would not be reproducible, and a lending decision that cannot be reconstructed months later is not defensible. The model, the calibrator and the policy decide; SHAP explains; Gemini writes. Nothing downstream of the policy can change what was decided."

3 Reading the factor contributions

SHAP TreeExplainer, computed per application — not global feature importance, which describes the model in general and says nothing about *this* applicant. The values are **additive in log-odds**:

$$\text{base_value} + \Sigma(\text{all SHAP values}) = \text{the model's raw score for this application}$$

That additivity is the point. The bars do not illustrate the explanation — they **are** the explanation, and they sum to the decision. Bars extending right raise risk; left lowers it.

One subtlety worth having ready: attributions are computed against the booster's *raw* output, not the calibrated probability. Isotonic calibration is monotonic, so it cannot reorder anything — explaining the raw score keeps the attributions exactly additive.

The live case, factor by factor

APP-31120547AC — REVIEW at 9.09%, actually fraud.

RAISING RISK

+1.1672	Name/email match 0.13
+0.9096	Device OS windows
+0.4924	Time at previous address not provided
+0.4908	Time at current address 60
+0.4603	Emails sharing this DOB 3

LOWERING RISK

-0.4451	Housing status BE
-0.3179	Declared income band 0.10
-0.2986	Banking relationship age 1.00
-0.2425	Application rate, 24h 2,834
-0.2062	Time since request started 0.02

The top factor scores **0.13 on name–email similarity** — the applicant's name barely matches their email. That alone contributes +1.17 log-odds, more than the next two combined. Three distinct emails share this date of birth, and no previous address was given: a coherent identity-fabrication signature, and it was in fact fraud.

TWO CAVEATS TO HAVE READY

The feature values look strange because BAF ships them pre-engineered. income = 0.10 is a normalised band, not rupees. name_email_similarity = 0.13 is a 0–1 score. Expect to be asked.

SHAP shows what the model learned, not what causes fraud. Note Application rate in the last 24 hours in the *protective* column. High velocity sounds suspicious, yet the model reads it as risk-reducing here — an interaction effect. It is a correlational attribution from a synthetic dataset, and exactly why a production deployment needs a disparate-impact review. Acknowledge it; do not explain it away.

Where the similarity panel fits

Separate from SHAP entirely. Weak matches mean "nothing on file resembles this strongly enough to act on" — stated explicitly rather than shown as an empty box, because an analyst cannot distinguish *no resemblance found* from *this feature is broken*. Only a **strong** match (≥ 0.60 , measured at $7.9\times$ lift) triggers escalation, and even then only APPROVE → REVIEW, never straight to BLOCK.

4 The live decision ledger

Applications replayed through the real scoring endpoint, one at a time, newest first.

Nothing here is simulated. Each row is a genuine HTTP call to `POST /api/v1/applications/score`, and the latency shown is what the server actually took.

LIVE DECISION LEDGER				replayed held-out data - true fraud rate 1.40%	
APPLICATION	P(FRAUD)	DECISION	OUTCOME	LATENCY	
APP-82EF7C9C63	0.05%	✓ APPROVE	CLEAN	73ms	
APP-68C1E8054C	0.03%	✓ APPROVE	CLEAN	75ms	
APP-6FC4DC8CB0	1.5%	✓ APPROVE	CLEAN	44ms	
APP-7C598DC6C2	0.03%	✓ APPROVE	CLEAN	52ms	
APP-4513FAB943	0.59%	✓ APPROVE	MISSED	45ms	
APP-E454D12A20	0.21%	✓ APPROVE	CLEAN	86ms	
APP-A3D2A2C265	0.76%	✓ APPROVE	CLEAN	48ms	
APP-3E26E043C5	0.21%	✓ APPROVE	CLEAN	81ms	
APP-01AF648A57	0.91%	✓ APPROVE	CLEAN	80ms	
APP-8E19A3509	0.91%	✓ APPROVE	MISSED	63ms	
APP-23C30B0A96	0.10%	✓ APPROVE	CLEAN	47ms	
APP-07A5A6FE7F	0.11%	✓ APPROVE	CLEAN	81ms	
APP-9A5A5093D9	5.7%	✗ REVIEW	NEEDLESS REVIEW	71ms	
APP-941E781E18	<0.01%	✓ APPROVE	CLEAN	35ms	
APP-EE003FF790	58%	✗ BLOCK	CAUGHT	39ms	
APP-565B44B84A	0.05%	✓ APPROVE	CLEAN	39ms	
APP-D0A982FC7	4.5%	✗ REVIEW	NEEDLESS REVIEW	91ms	
APP-CAEC724AA3	0.26%	✓ APPROVE	CLEAN	70ms	
APP-490842C191	0.05%	✓ APPROVE	CLEAN	68ms	
APP-0B3B51BF1E	2.6%	✓ APPROVE	MISSED	62ms	
APP-713B746644	<0.01%	✓ APPROVE	CLEAN	61ms	
APP-0A9CF33F19	1.4%	✓ APPROVE	CLEAN	84ms	
APP-6D1EE81272	0.05%	✓ APPROVE	CLEAN	57ms	
APP-994A199669	<0.01%	✓ APPROVE	CLEAN	57ms	
APP-F79CB8FFFB	9.1%	✗ REVIEW	TO ANALYST	34ms	
APP-E49406D14E	<0.01%	✓ APPROVE	CLEAN	31ms	
APP-4E54084101	0.49%	✓ APPROVE	CLEAN	37ms	
APP-E49FC4BABE	0.26%	✓ APPROVE	CLEAN	35ms	
APP-C1B5FFA750	1.4%	✓ APPROVE	CLEAN	52ms	

Application	The ID, with a thin left rule in the decision colour — lets the eye group the column without a second badge.
P(fraud)	Calibrated probability. Precision scales with magnitude (58.33%, 4.49%, <0.01%); a fixed decimal count would floor small values to "0%".
Decision	Colour <i>plus</i> glyph <i>plus</i> text. Roughly 1 in 12 men has a colour vision deficiency; encoding the verdict in hue alone would make this unusable for them.
Outcome	The demonstration affordance — see below.
Latency	Server-side, for that specific decision.

Streams at 900ms — fast enough to feel live, slow enough to read. Bounded at 60 rows so a long demo cannot grow the DOM without limit, and pinned to the top so the newest decision never scrolls out of view.

The Outcome column is the honest bit

Because held-out data is being replayed, the true label is known.

	ACTUALLY FRAUD	ACTUALLY GENUINE
BLOCK	caught	false positive
REVIEW	to analyst	needless review
APPROVE	MISSED	clean

Note the colour logic: BLOCK is red as a *decision* but **caught** is green as an *outcome*. Severity and correctness are different axes, and conflating them would mislead.

POINT AT A MISSED ROW

"That one was fraud and we approved it. The ledger shows misses, not just catches — a fraud console that only displays its wins is a sales pitch, and the first sharp question would dismantle it."

5 The metrics strip

SCORED 29 this session	BLOCKED 1 3 to review	FRAUD CAUGHT 40% 3 missed	MEAN LATENCY 59ms score + explain, server-side	BEHAVIOURAL SIGNAL 41% share of model gain	CALIBRATION ERROR 0.0028 from 0.135 uncalibrated
---	--	--	---	---	---

Six tiles — and the non-obvious thing is that **four are live session counters and two are fixed properties of the model**. Worth knowing before someone asks why two of them never move.

THE FOUR LIVE COUNTERS

Scored	Rows this session
Blocked	Count of BLOCK; hint shows the REVIEW count
Fraud caught	caught / (caught + missed)
Mean latency	Mean of latency_ms

THE TWO STATIC ONES

Behavioural signal 41%	Share of total model gain from velocity, device and session features
Calibration error 0.0028	Held-out ECE, from 0.135 before isotonic calibration

The static pair come from `metrics.json`, measured once on the test months. They are on screen because they are the two properties that most justify the design — not because they move.

"Why is accuracy not on the strip?"

At 1.4% prevalence, predicting "never fraud" scores 98.6%. Leading with accuracy here reports the base rate. Its absence is deliberate, and is itself a point worth making.

5 The metrics strip — three caveats

Each of these is a question waiting to be asked. Better to say it first.

1 — "FRAUD CAUGHT" COUNTS A REVIEW AS A CATCH

The arithmetic is $\text{caught} = \text{actual fraud AND decision} \neq \text{APPROVE}$. Routing a fraud to an analyst counts as caught, even though it is only a real catch if the analyst catches it — assumed at 90%. The tile is **optimistic by construction**. Better to say so than to be caught by it.

2 — THE NUMBER WILL LOOK BAD EARLY

A real 25-application sample gave **20% caught, 4 missed**, against a headline of 55.3% recall at a 5% alert rate. The denominator was five frauds; one more catch swings it to 40%. Let the stream run. If the number is ugly on camera, say "five frauds is not a sample" rather than looking uncomfortable.

3 — THE STREAM IS OVER-SAMPLED, AND DISCLOSES IT

The panel header reads *true fraud rate 1.40%* while the feed is stratified to **20% fraud**, one every five positions. At the natural rate a sixty-second demo would show roughly one fraudulent application, which demonstrates nothing.

Fraud caught is still valid — recall is $TP/(TP+FN)$, which does not depend on prevalence provided the fraud cases were randomly sampled. They were. **Blocked counts are not** — 1 block in 25 is an artefact of 20% fraud, not a production block rate.

"Are you inflating the numbers by over-sampling fraud?"

Every stream response returns `sampled_fraud_share`, `true_fraud_rate` and a written disclosure string, and the console prints the true rate on screen. Stated, not hidden — and recall is invariant to it.

6 The audit trail — lower-right tab

Everything the system decides, written to PostgreSQL and impossible to rewrite.

MODEL HEALTH		AUDIT TRAIL		193 written	
DECISIONS 1,016		AUDIT ROWS 1,021		FRAUD VECTORS 400	
Append-only — Postgres rejects UPDATE and DELETE on this table					
10:03:52 decision APP- E20B8D0E9B				REVIEW	
10:03:49 decision APP- 82EF7C9C63				APPROVE	
10:03:48 decision APP- 68C1E8854C				APPROVE	
10:03:48 decision APP- 6FC4DC8C8D				APPROVE	
10:03:47 decision APP- 7C59B0C6C2				APPROVE	
10:03:46 decision APP- 4513FAB943				APPROVE	
10:03:45 decision APP- L454D12A20				APPROVE	
10:03:44 decision APP- A3D2A2C265				APPROVE	
10:03:43 decision APP- 3E26E043C5				APPROVE	
10:03:42 decision APP- 01AF648A57				APPROVE	
10:03:41 decision APP- 8E159A3509				APPROVE	
10:03:40 decision APP- 232CB0BA96				APPROVE	
10:03:39 decision APP- 07A16AFE7F				APPROVE	
10:03:39 decision APP- 9AAC4073D9				REVIEW	

Three counters, an append-only notice, and a live tail of decisions and analyst verdicts.

Decisions	Rows in the decisions table
Audit rows	Entries in audit_log
Fraud vectors	Confirmed-fraud embeddings in pgvector — 400 seeded from history

The header badge reads postgres in green when recording is live. It turns amber or red if the write queue backs up or drops anything — a green light over a queue that is not draining would be worse than no light at all.

THE STRONGEST BEAT IN THE DEMO — DO NOT RUSH IT

"Every decision you just watched is in Postgres. The audit table is append-only, enforced by a database trigger rather than by convention — nothing in this application can rewrite a recorded decision. Now watch: I confirm one case as fraud, the index goes from 400 to 401, and the next application matching that pattern is escalated automatically. No retraining. And because it is in Postgres, it survives a restart."

The audit counters surviving a restart the process did not choose is the single clearest demonstration that persistence is real. If the servers get restarted during rehearsal, point at the row count afterwards.

DO NOT CONFIRM A CASE DURING REHEARSAL

Confirming a fraud writes a *permanent* vector. Doing it on an application that was actually genuine teaches the index that a genuine profile is fraudulent, and later lookalikes get escalated for nothing — which then shows up on camera as an unexplained NEEDLESS REVIEW. Clean up with:

```
python scripts/reset_demo.py --vectors-only --apply
```

Then restart the API so the mirror is rebuilt.